ログイン 新規登録

トレンド Qiita Engineer Festa 2024 質問 公式イベント 公式コラム キャリア・転職 NEW

この記事は最終更新日から5年以上が経過しています。

@smicle

三項演算子の適切な使い方（条件演算子）

JavaScript プログラミング 三項演算子 条件演算子

最終更新日 2018年12月07日 投稿日 2018年12月07日

皆さん三項演算子をif文の短縮形だと思っていませんか？
あくまでif文は文、三項演算子は演算子なので意味合いが違います。
三項演算子を適切に使用することで理解しやすいコードになるので、例題を見ながら
三項演算子の使い所について学んでいってください。
※ 使用する言語は javascript（ES2015）です。

三項演算子（条件演算子）とは

まず、三項演算子についてのおさらいです。MDN Web Docsより引用して説明します。

> 式と演算子・条件（三項）演算子
>
> 条件演算子はJavaScriptで唯一3つの被演算子を取る演算子です。
> 条件に基づいて2つの値のうち1つを選択します。
> 標準的な演算子を使用できる場所ならどこでも条件演算子を使用できます。

Qiitaにログインして、便利な機能を使ってみませんか？
あなたにマッチした記事をお届けします
便利な情報をあとから読み返せます
ログイン 新規登録 また後で

180 151

- `condition`
  - `true` か `false` かを評価する式です。
- `expr1, expr2`
  - 各々の値の場合に実行する式です。 `condition` が `true` の場合、演算子は `expr1` の値を返します。そうでない場合は `expr2` の値を返します。

ポイントは**条件に基づいて2つの値のうち1つを選択する**ところと、**演算子を使用できる場所ならどこでも使用できる**ところです。
単純にいうと、三項演算子は**真式の値を返し、演算子が使える場所全てで使える**ということです。

何もかんでもif文を三項演算子に置き換えれば良いわけではありません、適切な使い所があります。
if文と三項演算子を状況に応じて使い分けましょう。**むやみにif文を三項演算子に置き換えると却って分かりにくくなります。**

三項演算子が適切なパターン

適切に使用することで、if文よりも簡潔かつ分かりやすく書ける場合があります。

値の代入

```
if (str === 'piyo') {
  result = 'piyo！！'
} else {
  result = 'not piyo…'
}
```

Qiitaにログインして、便利な機能を使ってみませんか？
あなたにマッチした記事をお届けします
便利な情報をあとから読み返せます
ログイン 新規登録

180 151

```
result = str === 'piyo' ? 'piyo！！' : 'not piyo…'
```

三項演算子の**値を返す**、という特徴を利用し代入をします。
このように左辺に対して真式の値を代入してあげたほうが、簡潔かつ分かりやすくなります。
三項演算子を理解していない人の為に、コメントを書いておくとなお良いです。

引数の中で分岐

```
if (str === 'piyo') {
  console.log('piyo!!')
} else {
  console.log('not piyo…')
}
```

strの値によって出力が変わるコードです。このif文を三項演算子に変えてみます。

```
str === 'piyo' ? console.log('piyo!!') : console.log('not piyo…')
```

ただ単にif文を三項演算子に変えました。
ですが、これも三項演算子の適切な使い方とは言えません。

```
console.log(str === 'piyo' ? 'piyo!!': 'not piyo…')
```

三項演算子は**演算子を使用できる場所ならどこでも使用できる**、という特徴を利用し `console.log` の引数の中で分岐させています。
このように引数の中で分岐させてあげることで、簡潔かつ分かりやすくなります。

Qiitaにログインして、便利な機能を使ってみませんか？
あなたにマッチした記事をお届けします
便利な情報をあとから読み返せます
ログイン 新規登録

180 151

```
  if (v < 500) {
    return 0
  } else {
    return 1
  }
})(displaySize)
```

constは再代入が出来ないので、宣言時に代入を行う必要があります。
なのでif文の場合、関数を呼び戻り値に代入する値を設定しなければなりません。

```
let displaySize = 100
const DISPLAY_PATTERN = displaySize < 500 ? 0 : 1
```

三項演算子なら関数を作らなくて良いので、ひと目で理解出来るコードになります。
定数への代入に三項演算子を使用するのは適切な使い方です。

if文のが適切なパターン

逆にif文のが理解しやすい場合です。この場合は三項演算子を使わずif文で書きましょう。

分岐が複雑

```
if (str === 'piyo') {
  console.log('piyo!!')
} else if (str === 'koke') {
  console.log('koke!!')
} else if (str === 'hoho') {
  console.log('hoho!!')
```

Qiitaにログインして、便利な機能を使ってみませんか？
あなたにマッチした記事をお届けします
便利な情報をあとから読み返せます
ログイン 新規登録

180 151

```
console.log(
  str === 'piyo' ? 'piyo!!' :
  str === 'koke' ? 'koke!!' :
  str === 'hoho' ? 'hoho!!' :
  'not bird…';
)
```

確かに分からなくもないですが決して見やすいとも言えません。
基本的にコードは理解しやすい方が良いので、三項演算子の方が理解にくいのであれば、if文の方が適しています。

ネストが深い

```
if (str === 'piyo') {
  if (str === 'mofu') {
    console.log('mofu!!')
  } else {
    console.log('piyo!!')
  }
} else {
  console.log('not piyo…')
}
```

if文がネストしているコードです。このif文を三項演算子に変えてみます。

```
console.log(
  str === 'piyo' ?
    str === 'mofu' ? 'mofu!!' :
    'piyo!!' :
  'not piyo…'
)
```

Qiitaにログインして、便利な機能を使ってみませんか？
あなたにマッチした記事をお届けします
便利な情報をあとから読み返せます
ログイン 新規登録

```
console.log(str === 'mofu' ? 'mofu!!' : 'piyo!!')
```

180 151

```
  console.log('not piyo…')
}
```

書くとしても、ネストしているif文を三項演算子に変えた方が見やすいです。
今回の場合、三項演算子を使う必要はないので、普通にif文で書いたほうが簡潔です。

以上が例題でした。

後書き

かの有名なリーダブルコードにもこんなことが書いてありました。

- 三項演算子
  - 行数を短くするよりも、他の人が理解するのにかかる時間を短くする。
  - 基本的にはif/elseを使おう。三項演算子はそれによって簡潔になるときにだけ使おう。

結局一番伝えたかったのは、このことなんだと後書きを書いてて思いました。

世の中には、三項演算子の使用を禁止するコーディングルールが存在するらしいですが、それは違うと思います。
確かに三項演算子を闇雲に使うと見にくくなります。ですが、適切なタイミングで使うことでif文より理解しやすくなるのです。

「三項演算子と聞くだけで分かりにくいと決めつける人」「三項演算子でやたら短く書き自己満足している人」そういう人達が、この記事により三項演算子の考えを改めてくれれば嬉しいです。

参考

Qiitaにログインして、便利な機能を使ってみませんか？
あなたにマッチした記事をお届けします
便利な情報をあとから読み返せます
ログイン 新規登録

180 151

新規登録して、もっと便利にQiitaを使ってみよう

1. あなたにマッチした記事をお届けします
2. 便利な情報をあとで効率的に読み返せます
3. ダークテーマを利用できます

ログインすると使える機能について

新規登録

ログイン

コメント

この記事にコメントはありません。

いいね以上の気持ちはコメントで

ログイン 新規登録

How developers code is here.

Qiitaにログインして、便利な機能を使ってみませんか？

あなたにマッチした記事をお届けします

便利な情報をあとから読み返せます

ログイン 新規登録

180 151

プライバシーポリシー
ガイドライン
デザインガイドライン
ご意見
ヘルプ
広告掲載
公式コラム
アドベントカレンダー
Qiita 表彰プログラム
API
Qiita 人気の投稿
Qiita（キータ）公式

Qiita 関連サービス
Qiita Team
Qiita Zine
Qiita 公式ショップ
運営
運営会社
採用情報
Qiita Blog

Qiitaにログインして、便利な機能を使ってみませんか？
あなたにマッチした記事をお届けします
便利な情報をあとから読み返せます
ログイン
新規登録

180
151